

Name:徐靖甯

Date:09/12

Topic: Tuples, Lists, Aliasing, Mutability, and Cloning

1. Today’s Challenge

Core concept from today’s OCW lecture:

Lists are mutable, while tuples are immutable. Aliasing happens when two variables refer to the same object, while cloning creates a separate copy.

AI-generated coding challenge title: Shared Team Rosters

Challenge: Shared Team Rosters

Problem Statement

A school stores student teams as a list of lists.

Each inner list contains the names of students on one team.

You are given:

- `teams`: a list of team rosters
- `inactive`: a list of student names who have left the competition

Write a function that removes every inactive student from every team.

Your function must modify the original team rosters appropriately and return the number of students removed.

The challenge is to ensure that all teams are processed correctly, even when students are removed from a team while it is being examined.

Input Specification

1 teams

A list of lists of strings.

1 inactive

A list of strings.

Output Specification

Return a single integer:

1 number_removed

The total number of students removed from all teams.

Constraints

- `teams` contains at least 1 team.
- A team may be empty.
- Student names are strings.
- A student may appear multiple times.
- A student may appear in more than one team.
- You may assume `inactive` contains no duplicate names.

2. My Initial Approach — Before AI Help

I planned to create a nested list and assign it to another variable. Then, I would modify one of the elements and check whether the original list also changed. I thought I could use `copy()` to make another list and compare the results.

Before asking AI for hints, briefly describe how you planned to solve the problem.

My approach:

I create a nested list and assign it to another variable. Then, I would modify one of the elements and check whether the original list also changed. I thought I could use `copy()` to make another list and compare the results.

3. AI Tutor Help

Did you ask the AI Tutor for help?

☐ No — I solved it independently

☒ Yes — I received one or more hints

The most useful hint/question from AI was:

when a list contains another list, using `copy()` only creates a shallow copy, so the inner list can still be shared between the original list and the copied list.

It helped me realize that:

I realized that `copy()` only creates a shallow copy. If a list contains another list, the inner list can still be shared between the original and the copied list.

4. My Revision

Did you change your approach or code after interacting with AI?

☐ No

☒ Yes

What did you change, and why?

I changed my code to test both a normal list and a nested list. I also compared aliasing with shallow copying. This helped me understand that simply creating another variable does not make a new object, and `copy()` does not always completely separate nested data.

5. Verification

5. Verification

My final program:

- ☒ Passed the provided examples
- ☒ Passed additional edge cases
- ☐ Still has unresolved problems

One edge case I tested:

Input:

```
a = [[1, 2], [3, 4]]
```

```
b = a.copy()
```

```
b[0][0] = 99
```

Expected output:

```
a = [[99, 2], [3, 4]]
```

Actual output:

```
a = [[99, 2], [3, 4]]
```

6. One-Minute Reflection

What idea from the OCW lecture did you transfer to this new problem?

I transferred the idea of mutability and aliasing to understand why changing one list can sometimes affect another list.

One thing I understand better now:

I understand better that variables do not always store completely separate data.

Two variables can refer to the same object, and this can cause unexpected changes.

One thing I am still unsure about:

No